**UNIT - II:**

Agile Scrum Framework: Introduction to Scrum, Project phases, Agile Estimation, Planning game, Product backlog, Sprint backlog, Iteration planning, User story definition, Characteristics and content of user stories, Acceptance tests and Verifying stories, Project velocity, Burn down chart, Sprint planning and retrospective, Daily scrum, Scrum roles – Product Owner, Scrum Master, Scrum Team, Scrum case study, Tools for Agile project management.

**Agile Scrum Framework:** Scrum is a widely used Agile framework for managing complex projects, especially in software development. It helps teams collaborate, adapt to change, and deliver high-quality results efficiently.

- Breaks work into small iterations called sprints for step-by-step delivery.
- Promotes collaboration, continuous feedback, and improvement.
- Developed by Ken Schwaber and Jeff Sutherland in the early 1990s.

**Scrum Workflow:** The Scrum workflow shows how work moves across stages to deliver a usable product increment in each sprint. It ensures continuous feedback, collaboration, and improvement. Scrum is a framework for organizing and managing work. The Scrum framework is based on a set of values, principles, and practices that provide the foundation to which your organization will add its unique implementation of relevant engineering practices and your specific approaches for realizing the Scrum practices.

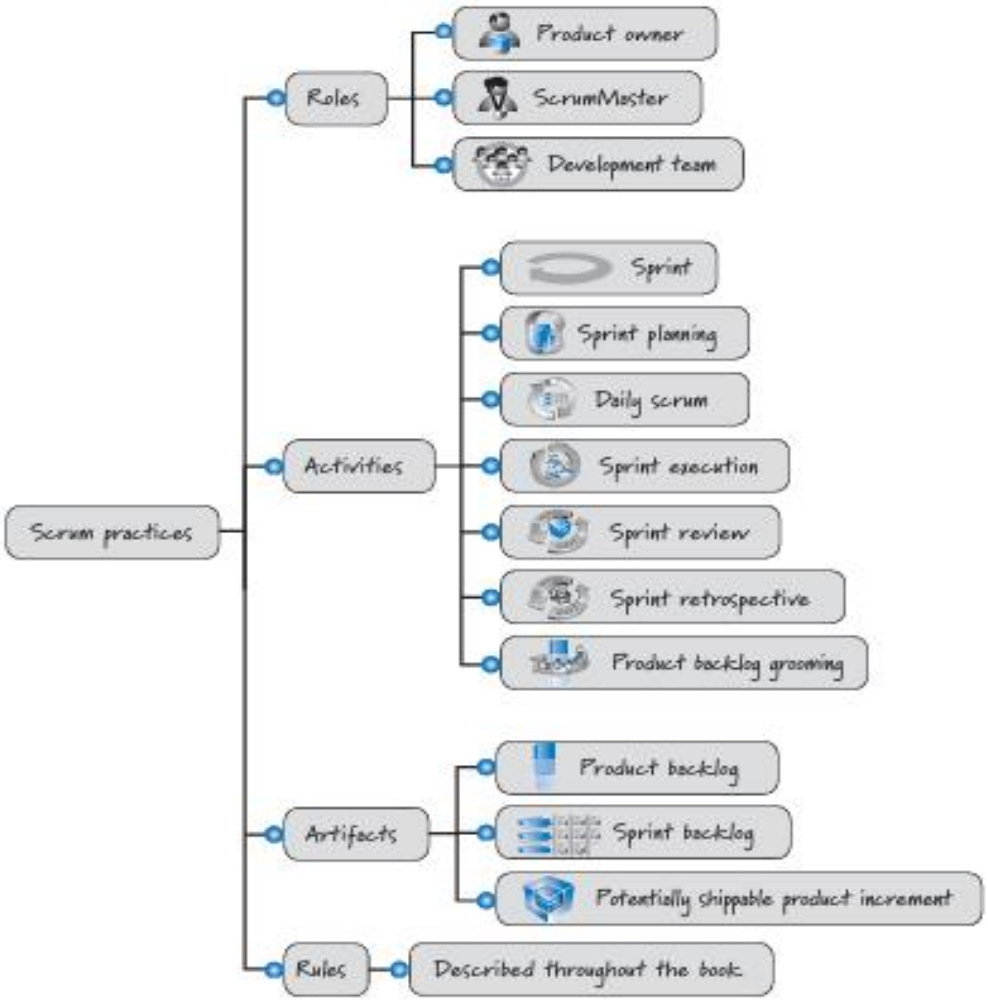



FIGURE 2.1 Scrum practices

**Scrum Roles:** Scrum development efforts consist of one or more Scrum teams, each made up of three Scrum roles: product owner, ScrumMaster, and the development team (see Figure 2.2). There can be other roles when using Scrum, but the Scrum framework requires only the three listed here




FIGURE 2.2 Scrum roles

The product owner is responsible for what will be developed and in what order. The Scrum Master is responsible for guiding the team in creating and following its own process based on the broader Scrum framework. The development team is responsible for determining how to deliver what the product owner has asked for.

**Product Owner**: The product owner is the empowered central point of product leadership. He1 is the single authority responsible for deciding which features and functionality to build and the order in which to build them. The product owner maintains and communicates to all other participants a clear vision of what the Scrum team is trying to achieve. As such, the product owner is responsible for the overall success of the solution being developed or maintained.

**Scrum Master:** The Scrum Master helps everyone involved understand and embrace the Scrum values, principles, and practices. She acts as a coach, providing process leadership and helping the Scrum team and the rest of the organization develop their own high performance, organization-specific Scrum approach. At the same time, the Scrum Master helps the organization through the challenging change management process that can occur during a Scrum adoption.

**Development Team:** Traditional software development approaches discuss various job types, such as architect, programmer, tester, database administrator, UI designer, and so on. Scrum defines the role of a development team, which is simply a diverse, cross-functional collection of these types of people who are responsible for designing, building, and testing the desired product.

**Scrum Activities and Artifacts:** Figure 2.3 illustrates most of the Scrum activities and artifacts and how they fit together

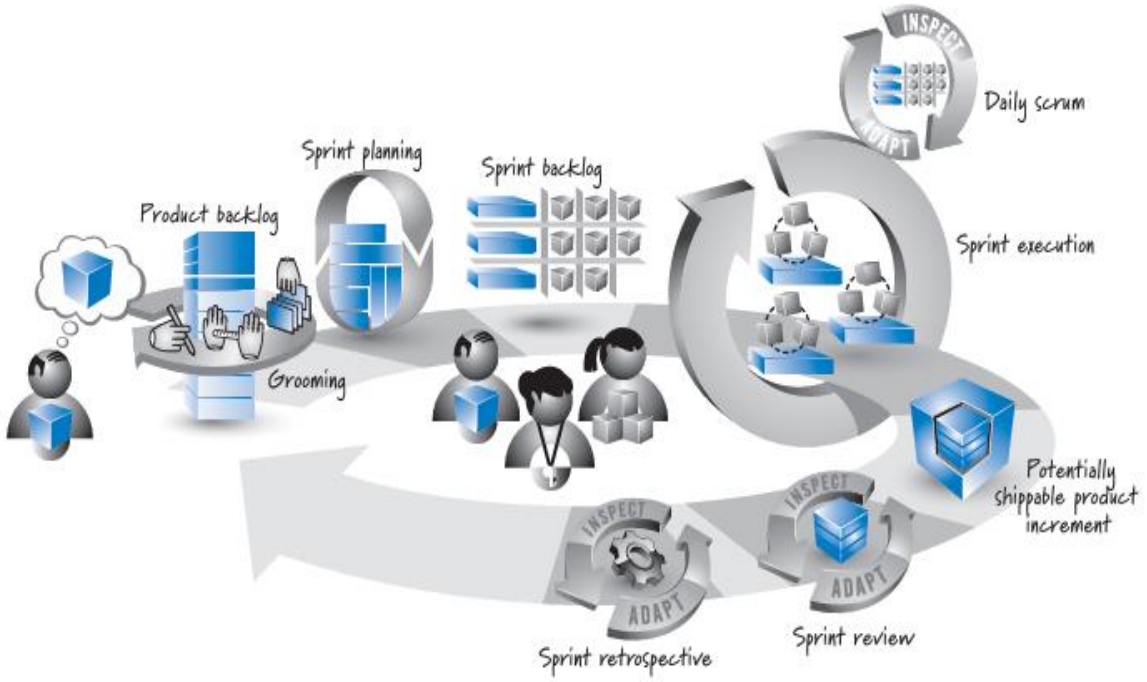



FIGURE 2.3 Scrum framework

Let's summarize the diagram, starting on the left side of the figure and working clockwise around the main looping arrow (the sprint). The product owner has a vision of what he wants to create (the big cube). Because the cube can be large, through an activity called grooming it is broken down into a set of features that are collected into a prioritized list called the product backlog. A sprint starts with sprint planning, encompasses the development work during the sprint (called sprint execution), and ends with the review and retrospective. The sprint is represented by the large, looping arrow that dominates the center of the fig ure. The number of items in the product backlog is likely to be more than a develop ment team can complete in a short-duration sprint. For that reason, at the beginning of each sprint, the development team must determine a subset of the product backlog items it believes it can complete—an activity called sprint planning, shown just to the right of the large product backlog cube.

**Product Backlog**: Using Scrum, we always do the most valuable work first. The product owner, with input from the rest of the Scrum team and stakeholders, is ultimately responsible for determining and managing the sequence of this work and communicating it in the form of a prioritized (or ordered) list known as the product backlog (see Figure 2.4). On new-product development the product backlog items initially are features required to meet the product owner's vision. For ongoing product development, the product backlog might also contain new features, changes to existing features, defects needing repair, technical improvements, and so on. The product owner collaborates with internal and external stakeholders to gather and define the product backlog items. He

then ensures that product backlog items are placed in the correct sequence (using factors such as value, cost, knowledge, and risk) so that the high-value items appear at the top of the product backlog and the lower-value items appear toward the bottom. The product backlog is a constantly evolving artifact. Items can be added, deleted, and revised by the product owner as business conditions change, or as the Scrum team's understanding of the product grows (through feedback on the software produced during each sprint).

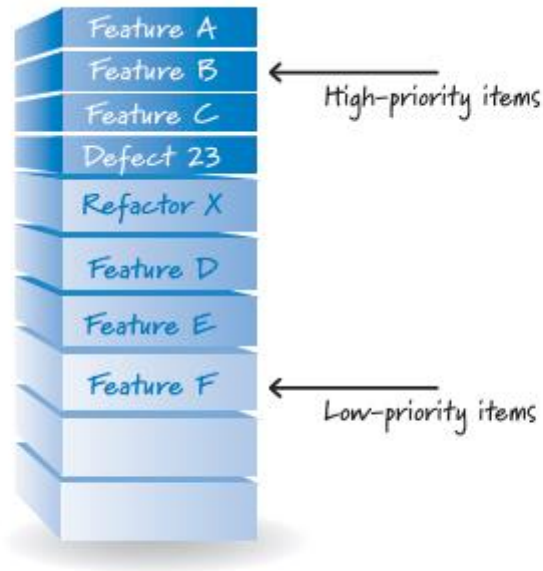



FIGURE 2.4 Product backlog

Overall the activity of creating and refining product backlog items, estimating them, and prioritizing them is known as grooming (see Figure 2.5).

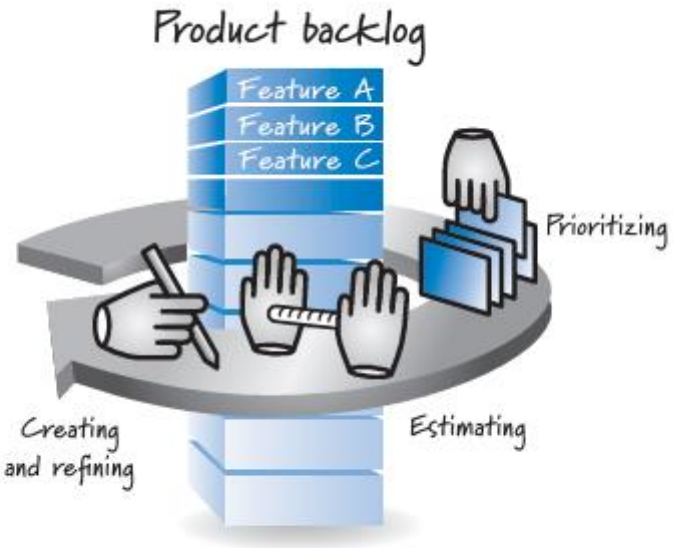



FIGURE 2.5 Product backlog grooming

**Sprints :** In Scrum, work is performed in iterations or cycles of up to a calendar month called sprints (see Figure 2.7). The work completed in each sprint should create

something of tangible value to the customer or user. Sprints are timeboxed so they always have a fixed start and end date, and gen erally they should all be of the same duration. A new sprint immediately follows the completion of the previous sprint. As a rule we do not permit any goal-altering changes in scope or personnel during a sprint; however, business needs sometimes make adherence to this rule impossible.




FIGURE 2.6 Product backlog item sizes

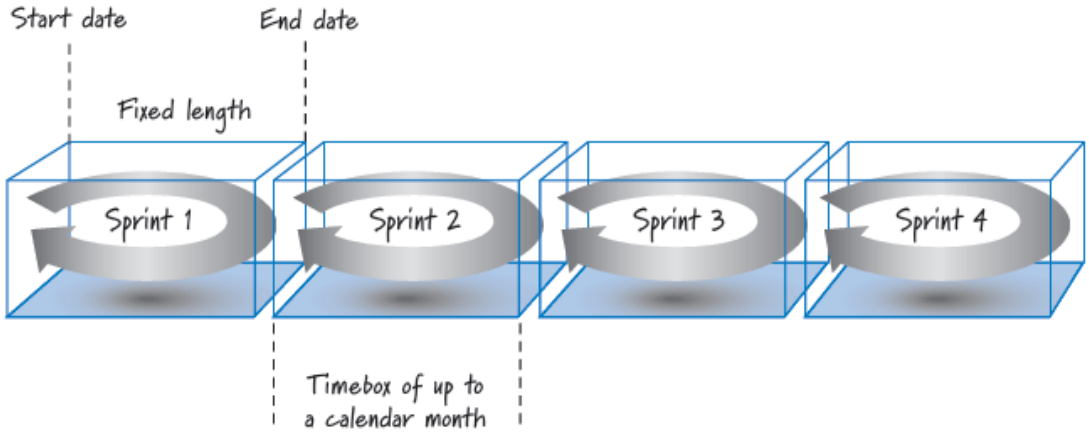



FIGURE 2.7 Sprint characteristics

**Sprint Planning**: A product backlog may represent many weeks or months of work, which is much more than can be completed in a single, short sprint. To determine the most impor tant subset of product backlog items to build in the next sprint, the product owner, development team, and ScrumMaster perform sprint planning (see Figure 2.8). During sprint planning, the product owner and development team agree on a sprint goal that defines what the upcoming sprint is supposed to achieve. Using this goal, the development team reviews the product backlog and determines the high priority items that the team can realistically accomplish in the upcoming sprint while working at a sustainable pace—a pace at which the development team can comfort ably work for an extended period of time.

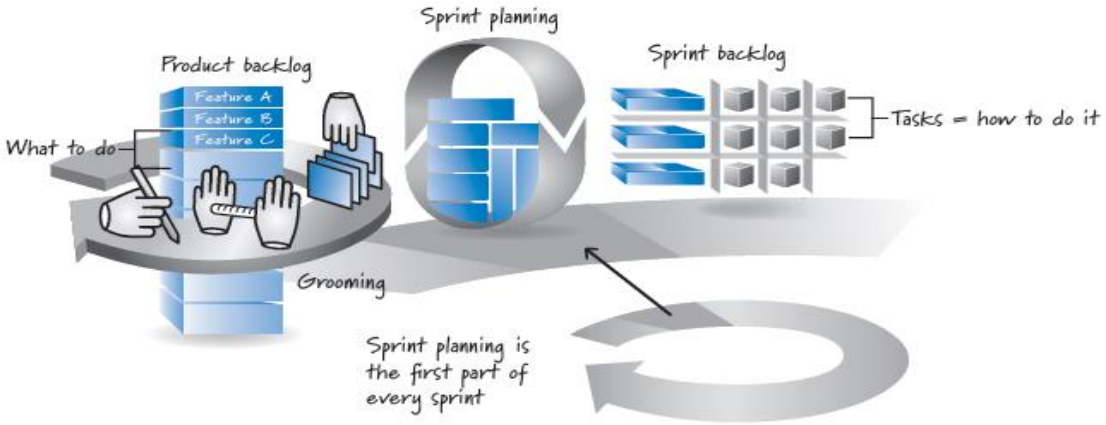



FIGURE 2.8 Sprint planning

To acquire confidence in what it can get done, many development teams break down each targeted feature into a set of tasks. The collection of these tasks, along with their associated product backlog items, forms a second backlog called the sprint backlog (see Figure 2.9).

The development team then provides an estimate (typically in hours) of the effort required to complete each task. Breaking product backlog items into tasks is a form of design and just-in-time planning for how to get the features done.

Most Scrum teams performing sprints of two weeks to a month in duration try to complete sprint planning in about four to eight hours. A one-week sprint should take no more than a couple of hours to plan (and probably less). During this time there are several approaches that can be used. The approach I use most often follows a sim ple cycle: Select a product backlog item (whenever possible, the next-most-important item as defined by the product owner), break the item down into tasks, and deter mine if the selected item will reasonably fit within the sprint (in combination with other items targeted for the same sprint). If it does fit and there is more capacity to complete work, repeat the cycle until the team is out of capacity to do any more work.

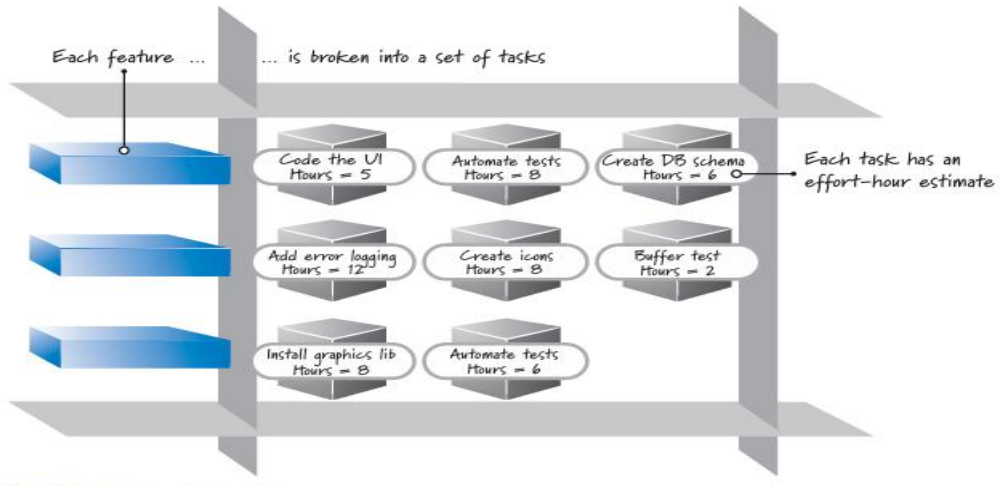



FIGURE 2.9 Sprint backlog

An alternative approach would be for the product owner and team to select all of the target product backlog items at one time. The development team alone does the task breakdowns to confirm that it really can deliver all of the selected product back log items.

**Sprint Execution:** Once the Scrum team finishes sprint planning and agrees on the content of the next sprint, the development team, guided by the ScrumMaster's coaching, performs all of the task-level work necessary to get the features done (see Figure 2.10), where "done" means there is a high degree of confidence that all of the work necessary for produc ing good-quality features has been completed.

**Daily Scrum** Each day of the sprint, ideally at the same time, the development team members hold a timeboxed (15 minutes or less) daily scrum (see Figure 2.11). This inspect-and adapt activity is sometimes referred to as the daily stand-up because of the common practice of everyone standing up during the meeting to help promote brevity.

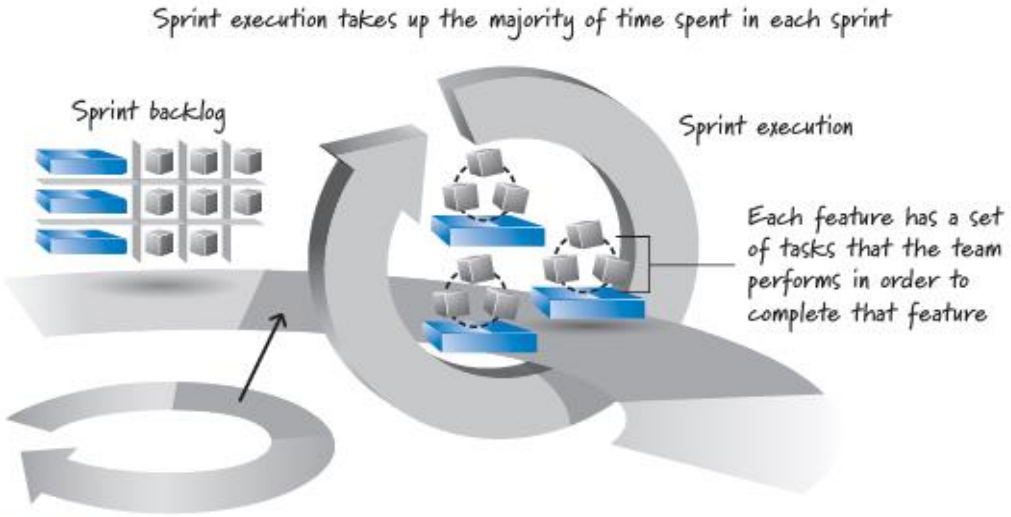



FIGURE 2.10 Sprint execution

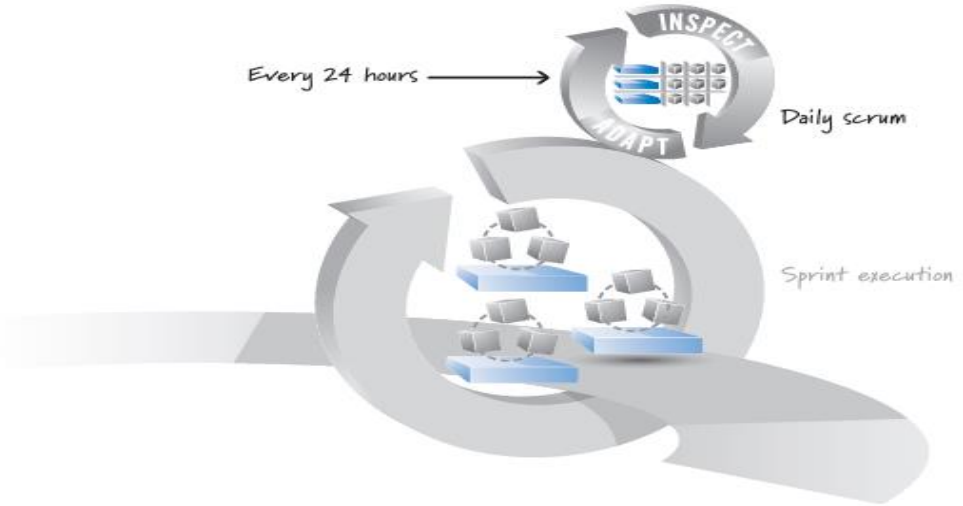



FIGURE 2.11 Daily scrum

**Sprint Review** At the end of the sprint there are two additional inspect-and-adapt activities. One is called the sprint review (see Figure 2.13).

The goal of this activity is to inspect and adapt the product that is being built. Critical to this activity is the conversation that takes place among its participants, which include the Scrum team, stakeholders, sponsors, customers, and interested members of other teams. The conversation is focused on reviewing the just-com pleted features in the context of the overall development effort. Everyone in atten dance gets clear visibility into what is occurring and has an opportunity to help guide the forthcoming development to ensure that the most business-appropriate solution is created.

A successful review results in bidirectional information flow. The people who aren't on the Scrum team get to sync up on the development effort and help guide its direction. At the same time, the Scrum team members gain a deeper appreciation for the business and marketing side of their product by getting frequent feedback on the convergence of the product toward delighted customers or users. The sprint review therefore represents a scheduled opportunity to inspect and adapt the product.

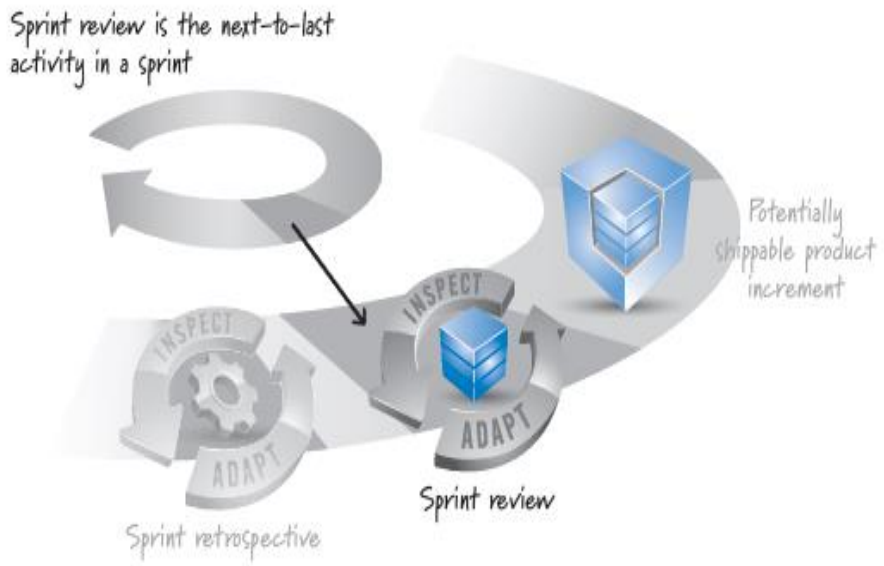



FIGURE 2.13 Sprint review

As a matter of practice, people outside the Scrum team can perform intra-sprint feature reviews and provide feedback to help the Scrum team better achieve its sprint goal.

**Sprint Retrospective:** The second inspect-and-adapt activity at the end of the sprint is the sprint retro spective (see Figure 2.14). This activity frequently occurs after the sprint review and before the next sprint planning.

Whereas the sprint review is a time to inspect and adapt the product, the sprint retrospective is an opportunity to inspect and adapt the process. During the sprint retrospective the development team, ScrumMaster, and product owner come together

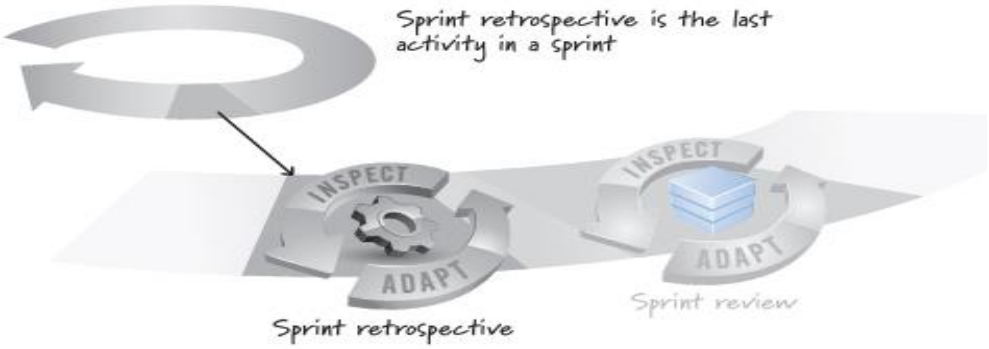



FIGURE 2.14 Sprint retrospective

to discuss what is and is not working with Scrum and associated technical practices. The focus is on the continuous process improvement necessary to help a good Scrum team become great. At the end of a sprint retrospective the Scrum team should have identified and committed to a practical number of process improvement actions that will be undertaken by the Scrum team in the next sprint.

After the sprint retrospective is completed, the whole cycle is repeated again—starting with the next sprint-planning session, held to determine the current highest value set of work for the team to focus on

**Project Phases:** Agile project management follows an iterative and incremental approach to software development. Unlike traditional methodologies, Agile does not divide the project into rigid sequential phases. Instead, development is carried out through multiple iterations (Sprints), with continuous customer feedback and improvement. Agile SDLC teams interact regularly through face-to-face meetings that promote problem-solving and communication. Teams can develop high-quality software that directly connects with customers by continuously reviewing and making adjustments because of continuous feedback loops. This iteration includes planning, carrying out, and reviewing each iteration to enable the product to adapt to changing market trends and customer needs. Agile flexibility enables development teams to respond swiftly to shifting objectives, resulting in software development that is more rapid and customer-focused.

**Stages of Agile SDLC (Software Development Life Cycle):**




Stages of Agile SDLC

**Stage 1: Concept**

Stakeholders and product owners work together to establish the project's goals and scope at the start of the Agile SDLC (Software development life cycle). To determine the project's feasibility, they look at requirements, expected expenses, and anticipated completion dates.

**Stage 2: Inception**

The inception phase is the second of the Agile SDLC (Software development life cycle's) six phases. The founder assigns positions, chooses the right team members, and provides development with the resources it needs throughout this stage.

It is crucial to create a plan and specify the core set of techniques and templates that will be applied to all future development work before starting the development phase. There are two parts to this planning phase, which is often referred to as the inception phase.

**Stage 3: Iteration**

UI/UX designers and developers must work closely together during this phase, which is usually the longest, to make sure that all business needs and feedback are implemented into the code. In order to finish the product backlog, the team goes through development iterations throughout this phase.

The agile methodology's iteration (or development) phase is essential because it allows the team to produce a product with the fewest features possible, with more functionality added later. It's time to finish the iteration, create technical documentation, and carry out quality assurance procedures when the development phase is over.

**Stage 4: Testing**

The team is pleased with the first iteration of the software, and development proceeded without a hitch. The app must pass a quality assurance review before being released. To make sure the app is fully working, the Agile team tests it by:

- Verifying the cleanness of the code
- Fixing flaws and mistakes
- Conducting trial runs

**Stage 5: Production**

- The Agile team releases the program to an on-premises server or the cloud when it's ready for public release.
- The product is live and available to clients as soon as it is launched. In the Agile SDLC, deployment is typically the most joyful phase: you made it! Give yourself a pat on the back now, but you still have one step left.

**Stage 6: Review:**

- At this stage of an Agile SDLC (software development life cycle), maintaining long-term success becomes more important than executing a successful launch. Following the product's successful debut, customers often offer suggestions for future features, interact with current upgrades, or provide feedback.
- The responsibility now shifts to the development and operations teams in addition to the stakeholders to continuously support the application to guarantee its smooth functioning.

- Each time a new feature is released, it needs to be tested alongside all existing features. There's a risk that newly written code will clash with previously written code with every compilation, breaking features and functionalities that are already in place. New features should improve the user experience, not make it worse.

**Project phases in scrum:** Scrum doesn't have rigid, sequential project phases. Instead, it relies on iterative learning cycles called Sprints. However, the framework is often broken down into five structured phases to manage the continuous flow of work from start to finish

- **Initiate -** This phase includes the processes related to initiation of a project: Create Project Vision, Identify Scrum Master and Stakeholder(s), Form Scrum Team, Develop Epic(s), Create Prioritized Product Backlog, and Conduct Release Planning.
- **Plan and Estimate -** This phase consists of processes related to planning and estimating tasks, which include Create User Stories, Estimate User Stories, Commit User Stories, Create Tasks, Estimate Tasks, and Update Sprint Backlog.
- **Implement** - This phase is related to the execution of the tasks and activities to create a project's product. These activities include creating the various deliverables, conducting Daily Standup Meetings, and refining (i.e., reviewing, fine-tuning, and regularly updating) the Product Backlog at regular intervals.
- **Review and Retrospect -** This phase is concerned with reviewing the deliverables and the work that has been done and determining ways to improve the practices and methods used to do project work.
- **Release -** This phase emphasizes on delivering the Accepted Deliverables to the customer and identifying, documenting, and internalizing the lessons learned during the project.

| Phase | Processes |
|---|---|
| **Initiate** | 1. Create Project Vision<br>2. Identify Scrum Master and Business Stakeholder(s)<br>3. Form Scrum Team<br>4. Develop Epic(s)<br>5. Create Prioritized Product Backlog<br>6. Conduct Release Planning |
| **Plan and Estimate** | 7. Create User Stories<br>8. Estimate User Stories<br>9. Commit User Stories |

| Phase | Processes |
|---|---|
| | 10. Create Tasks<br>11. Estimate Tasks<br>12. Update Sprint Backlog |
| **Implement** | 13. Create Deliverables<br>14. Conduct Daily Standup<br>15. Refine Prioritized Product Backlog |
| **Review and Retrospect** | 16. Demonstrate and Validate Sprint<br>17. Retrospect Sprint |
| **Release** | 18. Ship Deliverables<br>19. Retrospect Release |

**Agile estimation** is a way for teams to **predict the effort, time, or complexity** of completing work (like user stories or tasks) in Agile projects. Unlike traditional estimation (which tries to be exact), Agile estimation focuses on **relative sizing and collaboration** rather than precise numbers. Estimation includes four main factors - money, effort, resources, and time needed to build a specific system or product. There are many techniques available in today's world for doing estimations in an Agile Project. The main objectives for doing estimations include Relative Estimation, discussions to get more information on items whose estimations need to be done and ensuring the commitment and enthusiasm of the whole team towards the tasks assigned to them.

**Agile Project Estimation Techniques are:**

1. Dot Voting: In Dot Voting Techniques all the user stories along with their description are posted on the board. Each member put a dot in front of those stories that they consider most important. This way the stories are sorted according to their priorities. This is done to select the most important stories that should be taken forward.
2. T-Shirt Size: This technique helps in open and mutual collaborative discussions. In this technique, t-shirt sizes -XS (Extra Small), S (Small), M (Medium), L (Large), and XL (Extra Large) are used. User stories are given t-shirt sizes according to the member's understanding. This technique provides rough estimation very fast.
3. Planning Poker: Planning Poker is the most famous Estimation technique in Agile. This technique makes sure that every member participates in the estimation and shares his/her opinion. In this technique cards with numbers on them, are given to each member of the team. The Product Owner reads the story, after which every member has to hold the card showing the level of effort they will make for the user story. Discussion and Re-estimation go on until the whole team reaches a consensus.
4. The Bucket System: The Bucket System estimation technique is much faster than the Planning Poker technique. In this buckets are created with random values: 0, 1, 2, 3, 4,

5, 8, 13, 20, 30, 50. The whole team estimates the items by placing them in these "buckets". In the Bucket System Divide and Conquer approach is followed.

5. Large/Uncertain/Small: This technique is for doing rough estimation and it is simpler than the Bucket system technique. All the items are categorized in Large/Uncertain/Small. First simple user stories are chosen for estimation then more complex ones are taken. It's a good technique when comparable items are in the Product Backlog.
6. Relative Estimation: In this technique, teams estimate user stories according to one another rather than giving them exact numerical values. For instance, they can state, without assigning specific point values, that Story X is twice as complex as Story Y. This method prioritizes selection and ranking while simplifying estimation.
7. Story Points: In this estimation technique, user stories are given story points based on complexities, effort, and risk by teams. Story points are a metric without units and can stand in for any comparable value decided by the team. This method is adaptable and frequently used alongside Planning Poker.
8. Estimation Based on Velocity: This technique states the amount of work a team can accomplish in a sprint is measured by the team's velocity. Teams calculate the amount of user story points that can commit to in the following sprint using their average velocity over several sprints.

**Planning Game:** The **Planning Game** is one of the core practices of **Extreme Programming (XP)**, an Agile software development methodology. It is a collaborative planning process where customers and developers work together to determine what features will be developed and when they will be delivered. The Planning Game is a technique used to **prioritize requirements, estimate effort, and plan iterations** by balancing business value and technical feasibility.

## Objectives

- Understand customer requirements.
- Prioritize features based on business value.
- Estimate development effort.
- Create realistic release and iteration plans.
- Improve communication between customers and development teams.

## Participants

1. **Customer/Product Owner**
    - Defines requirements.
    - Prioritizes features.
    - Decides what should be developed first.
2. **Development Team**
    - Estimates effort required for each feature.
    - Provides technical insights and feasibility assessments.

**Phases of the Planning Game**

**1. Release Planning**

- Conducted at the beginning of a project.
- Customer writes requirements as **User Stories**.
- Developers estimate the effort for each story.
- Customer prioritizes stories based on business value.
- Team determines which stories can be delivered in each release.

**2. Iteration Planning**

- Conducted at the start of each iteration (sprint).
- Team selects user stories from the release plan.
- Tasks are broken down into smaller activities.
- Developers estimate task durations.
- Iteration goals and commitments are established.

**Steps in the Planning Game**

1. **Write User Stories**
   - Requirements are expressed as short, simple user stories.
2. **Estimate Effort**
   - Developers estimate story points or ideal development time.
3. **Prioritize Stories**
   - Customer ranks stories according to business value.
4. **Select Stories**
   - Based on team velocity and capacity, stories are chosen for the next iteration.
5. **Create Iteration Plan**
   - Tasks are identified and assigned.
6. **Review and Adjust**
   - Plans are revised based on feedback and changing requirements.

**Example:** Consider an online shopping application:

| User Story | Business Priority | Effort (Story Points) |
|---|---|---|
| User Login | High | 3 |
| Product Search | High | 5 |
| Shopping Cart | Medium | 8 |
| Wishlist | Low | 5 |

- The customer prioritizes **Login** and **Product Search**.
- The development team estimates effort.
- Based on available capacity, the first iteration includes Login and Product Search.

**Benefits**

- Encourages customer involvement.
- Improves estimation accuracy.
- Supports changing requirements.
- Delivers high-value features early.
- Enhances collaboration and transparency.

**Limitations**

- Requires active customer participation.
- Estimates may vary with team experience.

**Sprint backlog:** A sprint backlog is a prioritized list of tasks and user stories that a software development team commits to complete within a single sprint. During sprint planning, a backlog is created, which serves as a dynamic, actionable plan or to-do list that directs and prioritizes the team's work throughout the sprint.
Without a properly maintained backlog, you risk working on items that aren't relevant to your customers or the product roadmap. Working in smaller, timed sprints helps teams remain agile and responsive.

The team uses the product backlog and their current workload to determine which tasks are feasible to complete within the sprint's timeframe. Once a task is deemed feasible, it moves to the sprint backlog. If it's too large, it's broken into smaller tasks and executed across multiple sprints.

**Working of Product Backlog and Sprint Backlog Together**: The Product Backlog and Sprint Backlog are closely connected in the Scrum framework. The Product Backlog represents the complete list of work items for the product, while the Sprint Backlog contains a selected set of items taken from it for implementation in a sprint.

During Sprint Planning, the team selects high-priority items from the Product Backlog and moves them into the Sprint Backlog. The Development Team then breaks these items into smaller tasks and works on them during the sprint. This ensures that long-term product goals are gradually achieved through short, focused iterations.

**Product Backlog vs Sprint Backlog**

| Feature | Product Backlog | Sprint Backlog |
|---|---|---|
| Scope | Entire product requirements | Selected sprint work |
| Ownership | Product Owner | Development Team |
| Flexibility | Continuously updated | Fixed during sprint |
| Focus | Long-term vision | Short-term execution |

**Iteration planning:** The purpose of iteration planning is for the team to complete the set of top-ranked product backlog items. This commitment is time boxed based on the length of iteration and team velocity.

Who is Involved?

- Scrum Master – The scrum master acts as a facilitator for the agile delivery team.

- Product Owner – The product owner deals with the detailed view of the product backlog and their acceptance criteria.
- Agile Team – Agile delivery defines their tasks and sets the effort estimates required to fulfil the commitment.

**Prerequisites of Planning**

- Items in product backlog are sized and have a relative story point assigned.
- Ranking has been given to portfolio items by the product owner.
- Acceptance criteria has been clearly stated for each portfolio item.

**Planning Process:** Following are the steps involved in iteration planning –

- Determine how many stories can fit in an iteration.
- Break these stories into tasks and assign each task to their owners.
- Each task is given estimates in hours.
- These estimates help team members to check how many task hours each member have for the iteration.
- Team members are assigned tasks considering their velocity or capacity so that they are not overburdened.

**User story definition:** A user story is a short, simple description of a software feature written from the end user's perspective in Agile development**.** User stories capture what a user needs to do and why, following the format: "As a [persona], I want [need] so that [benefit]." They are the smallest unit of work in an Agile framework, focusing on delivering value to the customer. User stories are a key component of agile software development. They are short, simple descriptions of a feature or functionality from the perspective of a user. User stories are used to capture requirements in an agile project and help the development team understand the needs and expectations of the users.

**Here are some key characteristics of user stories:**

- User-centric: User stories focus on the needs of the user and what they want to achieve with the software.
- Simple: User stories are short and simple descriptions of a feature or functionality.
- Independent: User stories can stand on their own and do not rely on other user stories.
- Negotiable: User stories are open to discussion and can be refined and modified based on feedback from stakeholders.
- Valuable: User stories provide value to the user and the business.
- Estimable: User stories can be estimated in terms of time and effort required for implementation.
- Testable: User stories can be tested to ensure they meet the needs of the user.
- Prioritized: User stories are prioritized based on their importance to the user and the business goals.

- Iterative: User stories are developed iteratively, allowing for feedback and changes throughout the development process.
- Consistent: User stories follow a consistent format, making them easy to understand and work with.
- Contextual: User stories are written in a way that provides context to the development team, helping them understand the user's needs and goals.
- Acceptance criteria: User stories have clear and specific acceptance criteria that define when the story is considered "done" and ready for release.
- Role-based: User stories are written from the perspective of a specific user role, helping to ensure that the development team is building features that are relevant and useful to that user.
- Traceable: User stories are tracked and linked to specific features and functionality in the software, making it easy to trace back to the original user need.

| Aspect | Details |
|---|---|
| **Definition** | Short description of a feature from user's perspective |
| **Standard Format** | "As a [persona], I want [need] so that [benefit]" |
| **Purpose** | Capture user needs and deliver value incrementally |
| **Key Framework** | The 3 C's: Card, Conversation, Confirmation |
| **Quality Criteria** | INVEST: Independent, Negotiable, Valuable, Estimable, Small, Testable |
| **Best For** | Agile methodologies (Scrum, Kanban, XP) |
| **Written By** | Product Owner with input from team and stakeholders |
| **Size** | Completable within one sprint (typically 1-2 weeks) |

**Characteristics and content of user stories:** A **User Story** is a short, simple description of a feature or functionality written from the perspective of the end user. It helps the development team understand what value needs to be delivered.

**Characteristics of Good User Stories (INVEST Criteria)**

1. **Independent (I)**
   - The story should be self-contained and not heavily dependent on other stories.
2. **Negotiable (N)**
   - The details can be discussed and refined between stakeholders and the development team.
3. **Valuable (V)**
   - It must provide clear value to the customer or user.
4. **Estimable (E)**
   - The team should be able to estimate the effort required to implement it.
5. **Small (S)**
   - The story should be small enough to be completed within a sprint or iteration.

6. **Testable (T)**
    - Clear acceptance criteria should exist so that the functionality can be verified.

**Content of a User Story**
A user story typically contains the following elements:
### 1. Title
A brief name that summarizes the functionality.
**Example:** User Login
### 2. User Story Statement
Usually written in the format:
**As a** [type of user],
**I want** [some goal],
**So that** [some benefit/value].
**Example:**
As a customer,
I want to log in to my account using my email and password,
So that I can access my personal information securely.
### 3. Acceptance Criteria
Conditions that must be satisfied for the story to be considered complete.
**Example:**
- User can enter a valid email and password.
- Error message is displayed for invalid credentials.
- Successful login redirects the user to the dashboard.

### 4. Priority
Indicates the importance of the story (High, Medium, Low).
### 5. Estimation
Effort estimate assigned by the team (Story Points, Ideal Days, etc.).
### 6. Notes/Dependencies (Optional)
Additional information, constraints, or dependencies.

**Acceptance tests and Verifying stories:** Acceptance Tests in Agile Scrum
Acceptance tests are predefined conditions or criteria that a user story must satisfy to be considered complete and accepted by the Product Owner. They ensure that the developed functionality meets business requirements and user expectations.

**Characteristics of Acceptance Tests:**

- Derived from user stories and acceptance criteria.
- Written before or during development.
- Focus on business requirements rather than technical implementation.
- Measurable, clear, and testable.
- Used by developers, testers, and Product Owners to validate functionality.

**Example: User Story:**
*As a customer, I want to reset my password so that I can regain access to my account.*

**Acceptance Tests:**

1. User can request a password reset using a registered email address.
2. A reset link is sent to the registered email.
3. The reset link expires after 24 hours.
4. User can create a new password that meets security requirements.
5. User can log in successfully using the new password.

**Verifying Stories in Agile Scrum**
Verifying stories is the process of ensuring that a user story has been correctly implemented and satisfies all acceptance criteria before it is marked as complete.

**Steps in Story Verification:**

1. **Review Acceptance Criteria**
   - Ensure all specified requirements are addressed.
2. **Execute Acceptance Tests**
   - Test the functionality against the predefined acceptance tests.
3. **Check Definition of Done (DoD)**
   - Verify that coding, testing, documentation, and review activities are completed.
4. **Product Owner Validation**
   - Product Owner reviews the implemented feature and confirms it meets business needs.
5. **Sprint Review Demonstration**
   - The completed story is demonstrated to stakeholders for feedback.

**Benefits of Verifying Stories:**

- Ensures quality and correctness.
- Confirms business value delivery.
- Reduces defects and rework.
- Improves stakeholder satisfaction.
- Helps maintain transparency within the Scrum team.

**Relationship Between Acceptance Tests and Story Verification**

- **Acceptance Tests** define *what needs to be validated*.
- **Story Verification** is the process of *performing those validations* and confirming that the story is complete.

**Project velocity:** Project velocity tells you exactly how much work your team can realistically complete in a given time period, and yet most teams either measure it wrong or skip it entirely.
**What is project velocity?** Velocity in project management is a planning metric, not a performance score. Treating it as the latter is one of the most common mistakes agile teams make. Velocity tells you what your team can deliver in a sprint based on historical data, giving you a factual, repeatable basis for planning future work.

**Defining project velocity:** Project velocity is the total amount of work completed within a sprint or iteration, typically measured in story points or finished work items. It reflects how much scope a team can realistically deliver during a planning cycle based on actual outcomes rather than assumptions. Over time, sprint velocity helps teams establish predictable delivery patterns that support stronger sprint planning and backlog prioritization decisions.

**Where project velocity comes from**

The concept of velocity in project management originates from Agile delivery frameworks such as Scrum and Extreme Programming, where teams plan work in short iterations and track completed scope after each cycle. These frameworks introduced velocity as a practical way to evaluate delivery capacity using historical performance across sprints. Teams rely on this historical signal to guide planning conversations, improve estimation quality, and maintain sustainable delivery rhythms across releases.

**Common units used to measure project velocity**

Teams measure project velocity using units that reflect completed scope within each iteration.

Common approaches include:

- Story points
- Completed user stories
- Tasks
- Features

**Benefits of Velocity in Agile:** The benefits of velocity in Agile are numerous and crucial for effective project management:

- Predictability: Teams can dependably use their ability to complete tasks within each sprint by using velocity. Better planning and forecasting of project deadlines and releases are made possible by this predictability.
- Resource Allocation: Teams may more efficiently deploy resources and make sure they have the required personnel and expertise to finish scheduled work inside each sprint by knowing their velocity.
- Performance Measurement: For Agile teams, velocity is a crucial performance measure. It enables them to monitor their development over time, spot patterns, and evaluate their effectiveness and productivity.

**Common Risks of Velocity in Agile:** Agile velocity can occasionally lead to issues, such as teams concentrating too much on doing tasks quickly rather than performing them correctly. Estimates may not always be precise, which might cause misconceptions about the actual amount of work that can be completed. A team that tries to take on too much too soon risks losing focus on its priorities and experiencing members tired. It is essential that teams stay balanced, put quality first, and avoid using too much energy to accomplish unrealistic goals.

**Burn down chart:** Burndown chart is a major parameter used in agile software development and scrum to detect how much work remains to be completed. It is the graphical representation of showing the left-out portion of the task versus time. Generally, time is taken on the abscissa and left out work on ordinates. It is highly used when a project is going to be done. The company gets the knowledge of 'How the team members are working", and "Can determine the accomplishment of the task".

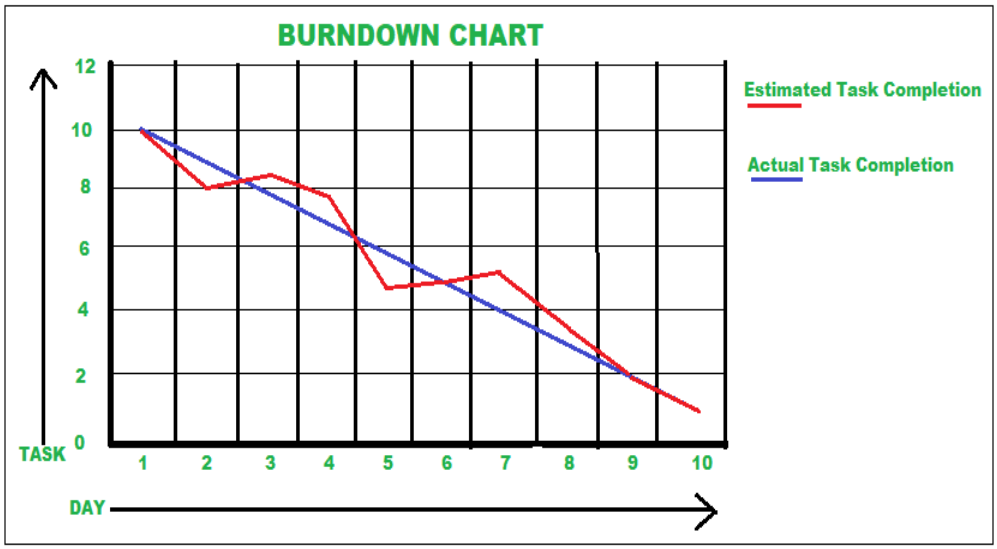
From the burn-down chart graph, we can estimate when the project is going to be complete. It is generally useful when a team does not make any progress in the project in the middle may be due to any reason, at that time they take the help of it and plan accordingly to complete the project within the given time. It checks the productivity of the work. It is widely used in agile and scrum project management.

**Steps to Create Burndown Chart:**

As from above, we got an idea that it is the graphical representation of work done, work to be done, and the time required for pending work done. So these are the main components that are involved during the burndown chart scrum creation steps. Let's know the steps individually.

- Estimating Work: Estimating work means from the backlog deciding what tasks need to be done and how much time is required to complete these tasks. More specifically how many user stories in how many days?
- Estimating Time: Estimating remaining time means how much time is left for a sprint.
- Estimating Effort: Estimating effort means managing effort accordingly looking at the work left and time left and tracking the burndown slope(means work completion track). In Y-axis a point(work needs to be done) and in X-axis a point(time remaining), the straight line from the Y-point to the X-point is the burn downslope.
- Tracking Progress: Tracking daily progress includes comparing the actual daily progress with the expected progress as per effort estimation. So if the daily progress line is below the burndown slope then the team is ahead of schedule, if the daily progress line is above the burndown slope then the team is far behind the target and if the daily progress line and burndown slope are lined up then it is in the right track.

Let's see an example to understand it more clearly.




In the above example, the blue line indicates the estimated task completion line and the orange line indicates the actual task completion line. According to the above Step:4, it can be determined when it was ahead of schedule, when it was far behind of target, and when it was on the right track.

**Uses:**

- The Burndown chart illustrates by what speed the team members are working to finish a given task by plotting user stories against time in a graph.
- First of all, an ideal straight line is drawn with a negative slope as a reference giving the inverse relationship between backlog (remaining work) and time.
- Every day during the daily scrum the team will discuss how many story points they have accomplished in the previous day. Then draw the line by removing the number of story points completed sprint.
- Burndown will help the team to see very quickly whether they are on track or not.
- If after a couple of days, the team sees that they are above the ideal line they need to find a way to come back on track. Otherwise, they will not be able to finish their user stories.
- If they go below the ideal line that means they might finish their user stories before the end of the sprint and they can ask the product's owner to take on another user story.
- It is never a straight line as the team can not work every day with the same velocity.

**Advantages of Burndown Chart:**

- We can see the total efforts a team puts against the given task from the graph.
- Burndown chart guides them to manage their tasks according to the time.
- Generally, software developers need a burn-down chart to estimate their efforts and pending works.
- It is useful as it provides insight into how the members are busy doing the work.
- Thus, every owner uses it to check the progress of the process of making a product.

- It works as a reminder and pokes the members if any project is delayed.
- It describes the period required to achieve the goal.
- It is a simple and easy way to check the progress of a project in scrum and agile management.
- Progress updates will keep the workers dedicated to their given tasks.
- Henceforth, it helps to finish everything before the deadline.

**Disadvantages of Burndown Chart:**

- In the burn-down chart, we can not see which user stories have been completed and whether that was the right user story or not. Only we can see the amount of work. So it is difficult to get a clear idea about specific user stories from the burn-down chart.
- The Burndown chart does not clarify the changes that occurred due to the increase or increase of any story points.
- From burn down, we can get the idea only on progress or lack but the progress is on the right track or not we can not get to know from it.

**Daily scrum: Daily Standup (or Daily Scrum)**

This is a short, daily meeting (usually 15 minutes or less) that occurs at the same time and place each day. The purpose is to keep the team aligned and informed about progress, identify any obstacles or issues, and ensure that everyone is on track to meet the sprint goal. Each team member answers three questions: What did I accomplish yesterday? What will I do today? Are there any obstacles or blockers in my way?

- Purpose of Daily Scrum: The Daily Standup is a short, daily meeting that keeps the team aligned and informed about progress, identifies any obstacles or issues, and ensures that everyone is on track to meet the sprint goal.
- Daily Standup (or Daily Scrum) Participants: The Scrum Master, Product Owner, and the entire development team participate in this meeting.
    - For the Scrum Master: Facilitate the meeting, ensure that the team stays focused on the three questions, and help resolve any issues or questions that arise.
    - For the Product Owner: Listen to the team's updates and provide any necessary feedback or clarification.
    - For the Development Team: Answer three questions: What did I accomplish yesterday? What will I do today? Are there any obstacles or blockers in my way?

**Agenda of Daily Scrum**

- Each team member answers three questions: What did I accomplish yesterday? What will I do today? Are there any obstacles or blockers in my way?

**When does of Daily Scrum ceremony happen?**

- When: Daily, at the same time and place each day.
- Frequency: Once per day, usually in the morning.
- Purpose: To keep the team aligned and informed about progress, identify any obstacles or issues, and ensure that everyone is on track to meet the sprint goal.

**Duration of Daily Scrum Ceremony**

- Duration: Usually 15 minutes or less.
- Frequency: Once per day, usually in the morning.

**Scrum roles:** Scrum defines three primary roles that work together to deliver a successful product:

**1. Product Owner (PO):** The **Product Owner** is responsible for maximizing the value of the product and managing stakeholder expectations. **The Product Owner is a key role in the Scrum development process.** They work closely with stakeholders to create a vision for the product and communicate this vision to the Scrum team. The Product Owner is responsible for ensuring the success of the project, managing and optimizing the product backlog, and making key decisions when needed.

A skilled product owner is crucial to achieving these goals. A Product Owner in a scrum project is responsible for defining product features and requirements that meet customer needs while also maximizing return on investment (ROI) for stakeholders.

**Key Responsibilities of a Product Owner**
The Product Owner is a champion of the product, guiding the development efforts and ensuring that the team works on the most valuable features. Some of the primary responsibilities of the Product Owner include the following:

- **Communicating the Vision:** The Product Owner works with stakeholders to create a vision for the product and communicates this vision to the Scrum team.
- **Managing the Product Backlog:** The Product Owner is responsible for managing the product backlog, which includes prioritizing needs and participating in Scrum meetings.
- **Making Key Decisions:** The Product Owner is expected to make decisions when the team needs them and actively participate in Scrum sprints.
- **Defining product requirements:** The Product Owner works closely with stakeholders to gather, refine, and determine the requirements for each Product Backlog Item, ensuring that they are well-formed, actionable, and testable.
- **Collaborating with the Scrum Team:** The Product Owner works closely with the Scrum Master and Development Team throughout the Sprint, providing guidance, answering questions, and making decisions on product functionality and priorities.
- **Accepting or rejecting completed work**: The Product Owner is responsible for reviewing and approving or declining the work completed by the Development Team, ensuring that it meets the acceptance criteria and provides value to the customers.
- **Stakeholder management**: The Product Owner acts as a middle layer between the Scrum Team and external stakeholders, ensuring that their needs and expectations are understood and addressed in the product development process.

## 2. Scrum Master

The **Scrum Master** is a servant leader who ensures that Scrum principles and practices are followed. A Scrum Master is a facilitator and coach who helps teams effectively adopt and

implement the Scrum framework. They ensure the team works efficiently, follows Agile principles, and continuously improves its processes to deliver value.

**Responsibilities**

**a) Scrum Process Facilitation**

The Scrum Master facilitates Scrum events such as:

- Sprint Planning
- Daily Scrum
- Sprint Review
- Sprint Retrospective

**b) Removing Impediments**

- Identifies issues affecting team productivity.
- Works with management and stakeholders to resolve problems.
- Ensures smooth project execution.

**c) Coaching the Team**

- Trains team members in Agile and Scrum practices.
- Encourages self-organization.
- Helps improve Agile maturity.

**d) Protecting the Team**

- Shields the team from unnecessary interruptions.
- Prevents scope creep during the sprint.
- Helps maintain focus on Sprint Goals.

**e) Promoting Continuous Improvement**

- Encourages learning and innovation.
- Facilitates retrospective discussions.
- Helps identify and implement process improvements.

**3. Scrum Team (Developers):** The Scrum Team consists of professionals who perform the work required to create a product increment. The team is self-organizing and cross-functional.

**Responsibilities**

**a) Sprint Planning**

- Participates in selecting Product Backlog items.
- Estimates effort required.
- Creates the Sprint Backlog.

**b) Development Activities**

- Design
- Coding
- Testing
- Integration
- Documentation

**c) Daily Scrum Participation**

- Discusses progress.
- Identifies obstacles.
- Plans work for the next 24 hours.

**d) Quality Assurance**

- Ensures adherence to coding standards.
- Conducts testing and reviews.

- Maintains product quality.

**e) Sprint Review and Retrospective**

- Demonstrates completed work.
- Collects feedback.
- Identifies improvement opportunities.

## Scrum Case Study: Case Study: Online Food Delivery Application Development Using Scrum

**Project Background:** A software company was assigned to develop an **Online Food Delivery Application** that allows customers to browse restaurants, place orders, track deliveries, and make online payments. The client wanted a working product quickly and expected frequent changes based on customer feedback. Therefore, the team adopted the **Scrum framework.**

### Scrum Team Structure

**Product Owner**: Responsible for defining requirements, prioritizing features, and maximizing product value.

**Scrum Master:** Facilitates Scrum events, removes impediments, and ensures Scrum practices are followed.

### Scrum Development Team

A cross-functional team consisting of:

- Front-end developers
- Back-end developers
- UI/UX designers
- Testers
- Database engineers

### Product Vision

Develop a user-friendly food delivery application that enables customers to order food efficiently and track deliveries in real time.

### Product Backlog

| Priority | User Story |
|---|---|
| High | As a customer, I want to register an account so that I can place orders. |
| High | As a customer, I want to search restaurants by location. |
| High | As a customer, I want to add food items to a cart. |
| High | As a customer, I want to make online payments securely. |
| Medium | As a customer, I want to track my order status. |
| Medium | As a restaurant owner, I want to update menu items. |
| Low | As a customer, I want to rate and review restaurants. |

**Sprint Planning**

**Sprint Duration**

2 Weeks

**Sprint 1 Goal**

Develop the basic customer registration and restaurant browsing functionality.

**Sprint Backlog**

- User registration
- User login
- Restaurant listing
- Search functionality
- Database setup

**Daily Scrum**

The team conducts a 15-minute meeting every day where members answer:

1. What did I complete yesterday?
2. What will I work on today?
3. Are there any impediments?

**Example**

**Developer:**

- Completed login module.
- Working on restaurant search feature.
- Waiting for API access from the database team.

The Scrum Master helps remove the dependency issue.

**Sprint Review**

At the end of Sprint 1, the team demonstrates:

- User registration
- Login functionality
- Restaurant browsing
- Search feature

**Stakeholder Feedback**

- Add filtering based on cuisine type.

- Improve search speed.
- Include restaurant ratings.

These suggestions are added to the Product Backlog.

**Sprint Retrospective**

The team discusses:

**What Went Well?**

- Good collaboration among developers.
- Sprint goal achieved on time.

**What Could Be Improved?**

- Better communication regarding API dependencies.
- Earlier testing of modules.

**Action Items**

- Introduce shared task tracking.
- Increase tester involvement from the beginning of the sprint.

**Sprint 2 Goal**

Develop ordering and payment functionalities.

**Sprint Backlog**

- Shopping cart
- Order placement
- Payment gateway integration
- Order confirmation notifications

**Benefits Achieved Through Scrum**

1. Faster delivery of working software.
2. Continuous customer feedback.
3. Better visibility of project progress.
4. Improved team collaboration.
5. Ability to adapt to changing requirements.
6. Higher customer satisfaction.

**Outcome**

After four sprints, the team successfully delivered the Online Food Delivery Application with:

- Customer registration and login
- Restaurant search and filtering
- Food ordering system
- Secure payment gateway
- Real-time order tracking
- Ratings and reviews

The project was completed with high customer satisfaction because Scrum enabled iterative development, continuous feedback, and rapid adaptation to changing business requirements.

**Key Scrum Artifacts Used**

- Product Backlog
- Sprint Backlog
- Increment

**Key Scrum Events Conducted**

- Sprint Planning
- Daily Scrum
- Sprint Review
- Sprint Retrospective

This case study demonstrates how Scrum helps teams deliver complex software projects incrementally while responding effectively to customer feedback and changing requirements.

**Agile project management tools** help teams plan, track, collaborate, and deliver projects efficiently while following Agile methodologies such as Scrum and Kanban.

# Popular Agile Project Management Tools

## 1. Jira Software

One of the most widely used Agile project management tools.

**Features:**

- Scrum and Kanban boards
- Sprint planning and backlog management
- Burndown and velocity charts
- Issue and bug tracking
- Custom workflows and dashboards

**Advantages:**

- Highly customizable
- Suitable for large and distributed teams
- Excellent reporting capabilities

## . Trello

A visual project management tool based on the Kanban methodology.

**Features:**

- Boards, lists, and cards
- Drag-and-drop interface
- Team collaboration
- Checklists and labels
- Workflow automation

**Advantages:**

- Easy to learn and use
- Ideal for small teams and projects

### 3.Azure DevOps

A comprehensive Agile and DevOps platform.

**Features:**

- Sprint planning
- Work item tracking
- Continuous Integration/Continuous Deployment (CI/CD)
- Source code management
- Test management

**Advantages:**

- Integrates development and project management
- Suitable for enterprise projects

### 4. Asana

A project and task management platform that supports Agile workflows.

**Features:**

- Task assignment and tracking
- Project timelines
- Custom workflows
- Team collaboration tools
- Reporting dashboards

**Advantages:**

- User-friendly interface
- Good for cross-functional teams

### 5. Monday.com

A flexible work management platform that supports Agile practices.

**Features:**

- Sprint management
- Kanban boards
- Automation
- Resource management
- Progress tracking

**Advantages:**

- Highly visual
- Easy customization

### 6. ClickUp

An all-in-one productivity and Agile project management solution.

**Features:**

- Sprint planning

- Task management
- Goal tracking
- Time tracking
- Agile dashboards

**Advantages:**

- Rich feature set
- Affordable for small and medium teams

### 7. VersionOne (Digital.ai Agility)

Designed specifically for Agile and scaled Agile environments.

**Features:**

- Portfolio planning
- Sprint tracking
- Agile reporting
- Team collaboration
- Release management

**Advantages:**

- Supports large Agile organizations
- Suitable for SAFe implementations

### 8. Rally Software (Broadcom Agile Operations)

An enterprise Agile management platform.

**Features:**

- Backlog management
- Release planning
- Team collaboration
- Agile metrics
- Portfolio management

**Advantages:**

- Strong support for large-scale Agile projects

## Comparison of Agile Tools

| Tool | Scrum Support | Kanban Support | Best For |
|---|---|---|---|
| Jira | Yes | Yes | Software Development Teams |
| Trello | Limited | Yes | Small Teams |
| Azure DevOps | Yes | Yes | Enterprise Development |
| Asana | Yes | Yes | Project Management |
| Monday.com | Yes | Yes | Team Collaboration |
| ClickUp | Yes | Yes | Startups and SMEs |
| VersionOne | Yes | Yes | Large Agile Organizations |
| Rally | Yes | Yes | Enterprise Agile Projects |